Lending Game – Stage 1 Strategy Note

Team "Lender 2" – Introduction to Finance for Data Scientists, HEC Paris

Internal note for the team's review before the October 17 submission

October 13, 2026

Summary

- We are lender 2. For the 100,000 new applicants we observe sex, employment, marital status, income and our private signal s_2; the competitors observe s_1 or s_3 instead. The past-loans file contains all three signals and the outcome, which lets us *measure* how much the competitors know that we do not, and *simulate* the whole market before submitting anything.
- Default is driven mostly by income and employment. Each private signal adds a little (AUC 0.717 $\rightarrow$ 0.735), the three together add more (0.760). The resulting hidden information has a standard deviation of 4.5 points of default probability – larger than any sensible margin – and is concentrated at low income (7.1 points at zero income, 0.3 points above 40k).
- Applicants take the cheapest offer, so we win a loan exactly when the competitors' signals were bad: the winner's curse. Naive pricing (break-even +3%) earns 1.7 M€ on average over twelve competitor scenarios and loses up to 2.2 M€.
- Our rule: break-even plus a 3–4% margin when our estimated PD is below an income-dependent threshold (8–12% below 20k of income, 25% above); otherwise an 89.8% "lender of last resort" offer, which is only ever accepted when nobody else lends. Average 6.9 M€, worst scenario +1.2 M€, market share about 22%.
- The submission contains an offer for every applicant: 54,418 normal offers between 3.0% and 17.6%, and 45,582 offers at 89.8%.

1 The game and what we optimize

Each of 100,000 applicants asks for a 10,000 € loan from three competing lenders. Lender k offers a rate $r(k, i)$ or nothing. One third of the applicants prefers each lender and is willing to pay 2% more to borrow from it: a type-k applicant takes the lowest of the three rates after subtracting 0.02 from lender k's rate. If the loan is repaid we earn $10{,}000\,r$; if the borrower defaults we lose 10,000 (zero recovery). Our score is total profit, judged in level and in rank against the two other teams (10% of the grade for stage 1, 20% for stage 2; the report is 70%).

With tens of thousands of loans, sampling noise is small (the standard deviation of total profit is about 0.5 M€ for 30,000 loans). The real uncertainty is strategic: what the other teams do. The stage-1 goal is therefore a strategy that is profitable against any plausible competitor, never loses money, and lets us learn the competitors' behaviour for stage 2, where the aggressive moves belong.

2 What the data says

Past loans: 100,000 rows, 9.95% defaults, no missing values; the new applications match the past population on every moment we checked (shares of employment status, income quantiles, signal distribution). Table 1 shows the main determinants of default.

Three facts matter for the strategy.

Income	Share	Default	Other variable	Share	Default
0	17.0%	20.2%	unemployed	14.3%	15.6%
1–5k	6.1%	15.1%	student	16.2%	12.9%
5–10k	29.9%	12.5%	retired	11.6%	9.0%
10–20k	25.9%	5.8%	employed	57.9%	7.9%
20–40k	12.8%	2.2%	$s_2 = 0$ ("no signal")	20.9%	9.7%
40–80k	5.1%	0.8%	$s_2 \in (0, 0.2]$	12.9%	14.9%
80k+	3.1%	0.1%	$s_2 \in (0.8, 1]$	15.4%	7.9%

Table 1: Default rates in the past loans. Women default slightly more than men (10.7% vs 9.4%) and singles more than married applicants (10.8% vs 9.4%).

- **Income dominates.** Default goes from 20% at zero income to 0.1% above 80k. The signals are weaker and non-linear: 15% default in the lowest bucket, flat around 8% above 0.6.
- **A signal of exactly zero means "no signal", not "worst score".** 21% of each signal is exactly 0, and those applicants default at the population average. A model that treats 0 as a very low score overprices a fifth of the applicants. Tree models isolate the spike on their own; we also add an explicit indicator.
- **The three signals are different information.** Their pairwise correlation is only 0.18 and the zeros are independent across signals. Hence the competitors genuinely know things about each applicant that we do not – the source of adverse selection in this game.

Model (out of sample, 5-fold)	AUC	log-loss	Income segment	$\sigma(U)$
X only (sex, employment, married, income)	0.717	0.2951	0	0.071
$X + s_2$ (us)	0.735	0.2908	1–5k	0.050
$X + s_1$ (lender 1)	0.735	0.2908	5–10k	0.052
$X + s_3$ (lender 3)	0.736	0.2904	10–20k	0.025
$X + s_1 + s_2 + s_3$ (full information)	0.760	0.2828	20–40k	0.008
			40k+	0.003

Table 2: Left: predictive power of the information sets. Right: standard deviation of the hidden information $U = p_{\text{full}} - \hat{p}_2$, i.e. how much the full-information default probability deviates from our own estimate. Overall $\sigma(U) = 0.045$.

Table 2 quantifies the asymmetry. Our model and the competitors' are equally good (same data, same method, symmetric signals), but a lender who saw all three signals would correct our estimate by ± 4.5 points of PD on average. At high income the competitors' signals are nearly worthless ($\sigma(U) = 0.3$ points); at zero income they move the PD by 7 points. This map tells us where we can compete and where we cannot.

3 The economics behind the pricing rule

3.1 Break-even rate

Interest is received only if the borrower repays, so the expected profit of a loan at rate r to a borrower with default probability p is $10{,}000\,[(1-p)\,r - p]$ and the break-even rate is

$$r^*(p) = \frac{p}{1-p}, \qquad \text{not } r = p. \tag{1}$$

The lecture formula NPV $= R - \mathbb{P}[\text{default}]$ assumes the interest is paid at issuance. A team applying it literally and pricing $r = p + m$ offers 23% to a 20%-PD applicant whose break-even is 25%; it wins those loans and loses money on them. We keep that behaviour as one of the competitor archetypes.

3.2 The winner's curse through the competitors' signals

Let $\hat{p}_2 = \mathbb{E}[D \mid X, s_2]$ be our estimate and $p = \mathbb{E}[D \mid X, s_1, s_2, s_3] = \hat{p}_2 + U$ the full-information probability, with $\mathbb{E}[U \mid X, s_2] = 0$. Our estimate is unbiased (Table 4), but the applicants who *accept* our offer are not a random sample: an applicant takes our offer when the competitors' effective rates are above ours, and competitors price higher when their signals are bad, i.e. when U is high. Writing $Q_i(r)$ for the probability that applicant i accepts our offer at rate r and $\pi_i(r) = \mathbb{E}[D_i \mid \text{accepts at } r]$ for the default rate of the accepting borrowers, the expected profit of the offer is

$$\Pi_i(r) = 10{,}000 \cdot Q_i(r) \left[(1 - \pi_i(r))\, r - \pi_i(r) \right], \qquad \pi_i(r) > \hat{p}_{2,i}. \tag{2}$$

This is the lecture's $M - \mathbb{E}[U \mid U > M - \bar{R}] < M$ with the acceptance event generated by the competitors' information rather than by the borrower's. Both Q_i and π_i depend on what the competitors do, which is why we simulate them instead of adding a fixed margin. The curse is measurable on the past loans: with naive break-even +3% pricing against competitors doing the same, the loans we win default at 10.0% while we predicted 8.1%; pricing *higher* makes it worse (12.1% realized vs 8.7% predicted at +5%), because a high price only wins the applicants everyone else found risky.

3.3 The 2% preference: fans versus everyone

For an applicant whose best competitor rate is c, our demand is a step function: we win our own fans (one third of applicants) if $r < c + 0.02$, and all three types if $r < c - 0.02$. Undercutting by 4 points therefore triples volume. Ignoring selection, the all-market price beats the fans-only price when $3[(1-p)(r-0.04) - p] > (1-p)\, r - p$, i.e. when

$$r > \frac{p}{1-p} + 0.06. \tag{3}$$

Grabbing market share pays only when the competitors sit more than 4 points above break-even, which teams equipped with a decent PD model will not do. The natural target in the safe segments is therefore our fans, at break-even plus a few points. Simulation confirms it: pricing at break-even +1% to gain share gives a 55% market share and a loss of 2 M€, and drags the competitors into losses as well.

3.4 The lender of last resort

Applicants have no outside option: whoever receives at least one offer borrows. An offer at a very high rate r_{net} is accepted only when every competitor's effective rate is above ours or absent, so it costs nothing when the competitors lend, and when both reject the applicant we lend at r_{net} to a borrower with default probability $d = \mathbb{E}[D \mid \text{both reject}]$. The loan is profitable if

$$(1-d)\, r_{\text{net}} - d > 0 \iff r_{\text{net}} > \frac{d}{1-d}. \tag{4}$$

Table 3 evaluates the abandoned pool on the past loans for competitors who reject above a PD threshold: as long as they reject at 25% or below, any rate above 74% is profitable; if they reject only above 30% the pool is unprofitable at any rate but represents 0.2% of applicants (a loss of about 0.05 M€ at worst). This "safety net" replaces "no offer" everywhere in our submission.

4 Methodology

4.1 Default probability models

Gradient-boosted trees (`HistGradientBoostingClassifier`) on employment dummies, marital status, income, log(1+income), the signal and a "signal = 0" indicator. Hyper-parameters were chosen

Competitors reject if PD >	Pool	Default d	Break-even $d/(1-d)$	Profit per loan (€) at a net of			
				0.989	0.90	0.80	0.70
15%	14.4%	24.3%	32%	5,061	4,387	3,630	2,873
20%	5.3%	29.9%	43%	3,934	3,311	2,610	1,910
25%	0.8%	42.6%	74%	1,420	909	335	−240
30%	0.2%	51.2%	none	−289	−724	−1,212	−1,700

Table 3: The applicants both competitors would reject (their own PD estimate above the threshold), on the past loans.

by 3-fold cross-validated log-loss (the right metric, since we need calibrated probabilities, not just ranking): learning rate 0.05, 7 leaves, 200 iterations, at least 200 observations per leaf, $L_2 = 1$. Four models are fitted with the same recipe: ours ($X + s_2$), each competitor's ($X + s_1$, $X + s_3$) and the full-information model ($X + s_1 + s_2 + s_3$). All predictions used in the simulations are 5-fold out-of-sample, so no lender is clairvoyant in the simulator; the model used to price the new applications is refitted on all 100,000 past loans.

Decile of $\hat{p}_2$	1	2	3	4	5	6	7	8	9	10
Predicted	0.007	0.022	0.035	0.054	0.077	0.101	0.123	0.145	0.179	0.253
Realized	0.006	0.021	0.033	0.053	0.077	0.102	0.122	0.143	0.182	0.256

Table 4: Calibration of our model (out of sample).

Sex is excluded from the final model: with or without it the out-of-sample AUC is 0.7349 and the log-loss 0.2908. It adds nothing, and pricing on sex would be illegal for a real lender, so we drop it at zero cost.

4.2 Market simulator

`lending_game/simulator.py` implements the rules literally: applicant types are drawn uniformly at random (the rules state one third each), the 2% preference is subtracted from the preferred lender's rate, the lowest effective rate wins, ties are broken at random, and the profit is computed from the realized defaults of the past loans. We also compute an "expected" profit using the full-information PD instead of the realized outcome: it has the same mean (the full-information model is calibrated) but much less noise, so it is the objective of the optimizer, while the realized profit is reported as a check. Every result is averaged over three independent type draws.

4.3 Competitor scenarios

We expect the other teams to work with the same data and similar boosted-tree models, so no edge can come from the model itself; the uncertainty is in their pricing rule. Each archetype in Table 5 maps a competitor's own PD estimate to a rate, and twelve scenarios combine them: eight symmetric (`base/base`, `aggressive/aggressive`, `cautious/cautious`, `no_reject/no_reject`, `slide/slide`, `slide_norej/slide_norej`, `smart/smart`, `base_net/base_net`) and four mixed (`base/aggressive`, `cautious/slide`, `smart/base`, `loose/tight`).

4.4 Our pricing rule and its optimization

For applicant i with our PD estimate $\hat{p}_i$ and income segment s_i,

$$r_i = \begin{cases} \dfrac{\hat{p}_i}{1-\hat{p}_i} + m(s_i) & \text{if } \hat{p}_i \le \tau(s_i), \\ r_{\text{net}} & \text{otherwise.} \end{cases} \tag{5}$$

Archetype	Rate	Rejects if PD >	Instead of "no offer"
base	$p/(1-p)+3\%$	25%	–
aggressive	$p/(1-p)+1\%$	never	–
cautious	$p/(1-p)+5\%$	20%	–
no_reject	$p/(1-p)+3\%$	never	–
loose / tight	$p/(1-p)+2\%$ / $+4\%$	30% / 20%	–
slide / slide_norej	$p+3\%$ (lecture formula)	25% / never	–
smart	$p/(1-p)$ + 2–6% rising as income falls	25%	0.99 safety net
base_net	$p/(1-p)+3\%$	25%	0.99 safety net

Table 5: Competitor archetypes, each applied to the competitor's own out-of-sample PD estimate.

The parameters are a margin m and a threshold τ per income segment (six segments: 0, 1–5k, 5–10k, 10–20k, 20–40k, 40k+) and the net rate. They are chosen by coordinate descent on grids (margins 0.5–10%, thresholds 4–100%, net in $\{\text{none}, 0.5, \ldots, 0.989\}$) to maximize the expected profit averaged over the twelve scenarios and three type draws, evaluated on the past loans. A richer rule with margins by segment × risk tercile improved the objective by only 0.2% and was discarded. The net rate was then fixed by a dedicated analysis (Section 5.5).

5 Results

5.1 The chosen rule

Segment	New applicants	Mean $\hat{p}$	Margin m	Threshold τ	Offered the net	Median normal rate
0	17,159	0.201	4.0%	12%	92%	17.4%
1–5k	6,142	0.150	3.5%	12%	74%	15.7%
5–10k	29,870	0.126	3.5%	10%	69%	12.3%
10–20k	25,766	0.058	3.0%	8%	18%	8.0%
20–40k	12,829	0.023	3.0%	25%	0%	5.3%
40k+	8,234	0.006	3.0%	25%	0%	3.4%

Table 6: Optimized parameters and their effect on the new applications; $r_{\text{net}} = 0.898$.

The optimizer's answer is simple to state: *compete normally where the competitors cannot know more than us; where they can, either take the clearly safe applicants or be the lender of last resort – never in between.* Above 20k of income the thresholds are high and we lend to almost everyone at break-even +3%. Below 20k, any applicant whose PD exceeds 8–12% gets the net. The margins are not lower at high income because, with the 2% preference, cutting the margin from 3% to 1% does not win any non-fan and loses two thirds of the profit on the fans (Section 3.3).

5.2 Comparison with naive strategies

Strategy (realized profit on the past loans, 12 scenarios)	Mean	Worst scenario	Mean share
Break-even +3%, lend to everyone	1.7 M€	−2.2 M€	32%
Break-even +3%, no offer if PD > 25%	2.3 M€	−1.9 M€	32%
Break-even +3%, net at 0.989 instead of no offer	2.9 M€	−1.9 M€	32%
Optimized rule (net 0.898)	**6.9 M€**	**+1.2** M€	22%

Table 7: Each strategy against the same twelve competitor scenarios.

Scenario (lender 1 / lender 3)	Our share	Our profit	$\hat{p}_2$ on won	Realized default	Lender 1	Lender 3
base / base	21.7%	5.4	0.054	0.068	3.2	3.2
aggressive / aggressive	9.1%	1.2	0.049	0.064	−4.0	−4.0
cautious / cautious	42.7%	27.3	0.066	0.076	6.5	6.2
no_reject / no_reject	20.8%	4.7	0.046	0.053	2.3	2.5
slide / slide	19.4%	4.7	0.051	0.067	−1.1	−0.9
slide_norej / slide_norej	18.6%	4.0	0.043	0.050	−2.2	−1.9
smart / smart	24.7%	6.8	0.057	0.067	6.2	6.3
base_net / base_net	21.7%	5.4	0.054	0.068	3.2	3.2
base / aggressive	12.6%	2.2	0.050	0.061	−1.1	−2.1
cautious / slide	27.2%	9.6	0.056	0.071	0.3	2.7
smart / base	23.0%	5.9	0.055	0.068	3.3	5.5
loose / tight	20.0%	5.3	0.054	0.067	2.6	1.9

Table 8: Optimized rule: realized profits in M€, averaged over three type draws.

5.3 Scenario by scenario

We are profitable in every scenario and first in all but one (`smart/smart`, where both competitors reason like us and keep their own 10–25% PD business at fat margins; we are then a close third with 6.8 M€ against 6.2–6.3 M€). When the competitors are aggressive they lose 4 M€ each while we keep +1.2 M€. The curse is tamed but visible: the loans we win default about 1.4 points more than we predict (6.8% vs 5.4% in the base scenario), against 2 points with naive pricing. The 27 M€ of the cautious scenario is the net at work on a 5% abandoned pool; we do not count on it.

5.4 Where the profit comes from

Segment	Applicants	Share won	Mean rate of won loans	$\hat{p}_2$ on won	Realized default	Profit
0	17,035	7.9%	86.3%	0.180	0.298	0.56 M€
1–5k	6,125	12.5%	73.2%	0.106	0.112	0.28 M€
5–10k	29,904	14.4%	65.6%	0.080	0.100	0.72 M€
10–20k	25,923	31.9%	23.0%	0.046	0.053	1.73 M€
20–40k	12,780	33.9%	5.4%	0.022	0.023	1.24 M€
40k+	8,233	33.2%	3.6%	0.006	0.007	0.80 M€

Table 9: Optimized rule in the base scenario (one type draw). Above 10k of income we win one third of the applicants – our fans – with no curse to speak of; below 10k most won loans are net loans to abandoned applicants, which default at 30% but are priced at 89.8%.

5.5 Robustness checks

The thresholds. Raising the thresholds to "look more moderate" is expensive (Table 10): at $\tau = 25\%$ everywhere the mean falls from 7.0 to 3.3 M€ and the worst scenario becomes a loss of 1.1 M€. The low thresholds are not an artefact of the scenario where both competitors reject a lot: re-optimizing without that scenario, only against competitors who never reject anyone, or only against rejecting competitors, returns *identical* thresholds. What drives them is the curse itself: at low income, the 10–25% PD band is where the competitors' hidden information is largest, and competing for it at any normal rate loses money. Removing the net altogether (plain "no offer") keeps the worst case but costs 2.4 M€ on average.

The net rate. The optimizer picks the highest allowed net (0.989) because none of the twelve scenarios has a competitor net below 0.99. If a competitor also thinks of the net, the 2% preference creates steps: against a competitor net at x we win everything abandoned if ours is below $x - 0.02$, only our fans if within ± 0.02, and nothing above $x + 0.02$. Table 11 evaluates our net against

Variant (same margins)	Mean expected profit	Worst scenario	Base scenario
Thresholds as chosen	7.0 M€	+1.2 M€	5.7 M€
$\tau = 15\%$ everywhere	6.3 M€	+0.2 M€	5.0 M€
$\tau = 20\%$ everywhere	5.0 M€	−0.5 M€	3.9 M€
$\tau = 25\%$ everywhere	3.3 M€	−1.1 M€	2.7 M€
$\tau = 30\%$ everywhere	2.5 M€	−1.3 M€	2.1 M€
Chosen thresholds, no offer instead of the net	4.6 M€	+1.2 M€	4.6 M€

Table 10: Sensitivity to the rejection threshold.

thirteen competitor cases (no net; both netting at 1.00, 0.99, 0.95, 0.90, 0.85, 0.80, 0.70; one netting at 0.90; cautious and loose rejecters with a 0.99 net; two non-rejecting cases). Lowering our net from 0.989 to 0.90 costs 0.14 M€ if nobody else nets and gains 1.4 M€ on average if somebody does; below 0.85 the lost margin outweighs the protection, and below 0.70 the net starts winning the very worst borrowers from competitors who lend to everyone at 70–85%. We chose **0.898**: it beats a competitor at 1.00, 0.99 or 0.95 outright, splits with one at 0.90, and is one tick below 0.899 in case someone has the same idea.

Our net rate	Mean (13 cases)	If nobody nets	If somebody nets
0.989	5.4 M€	4.0 M€	5.8 M€
0.949	6.5 M€	3.9 M€	7.3 M€
0.899	6.4 M€	3.9 M€	7.2 M€
0.849	6.3 M€	3.8 M€	7.0 M€
0.799	6.0 M€	3.7 M€	6.8 M€
0.699	5.5 M€	3.5 M€	6.1 M€
No offer	4.6 M€	3.5 M€	5.0 M€

Table 11: Expected total profit as a function of our net rate, margins and thresholds held fixed.

6 The submission

`output/round1/rates_round1.csv`: 100,000 rows, columns `id` (1 to 100,000, in order) and `rate`, no missing value, maximum 0.898. 54,418 normal offers (7,270 below 4%; 13,250 at 4–6%; 11,382 at 6–8%; 14,389 at 8–12%; 8,127 at 12–17.6%) and 45,582 offers at 0.898. No rate is a round number, to avoid ties with competitors who use them.

7 Assumptions, risks and what could go wrong

- **Competitor behaviour is a guess.** The twelve archetypes span cautious to aggressive and include the lecture-formula mistake and selection-aware teams, and we optimized the average while checking the worst case; but a behaviour outside this family (for instance random rates, or a flat 10% for everyone) was not simulated. The rule stays profitable in every scenario we tried; the main risk is *lower* profit, not a loss.
- **The net earns nothing if the competitors lend to everybody**, and nothing if they both net below 0.878; neither case costs us money. It earns 0.7 M€ in the base scenario and far more if they reject at 20% or below. Its expected value rests on the abandoned pool defaulting below 47%, which holds for any rejection threshold up to 25% and fails only for a 0.2% pool.
- **Our thresholds give up share.** We lend to 9–22% of applicants depending on the scenario; above 10k of income we win our full third, below we deliberately walk away from the risky middle. This is a choice, not an accident.

- **The competitors' models are assumed as good as ours.** If theirs are worse, the curse is smaller and their rejections noisier (better abandoned pool); if better, the curse is larger and the low thresholds protect us.
- **Types are uniform and independent of characteristics**, as stated in the rules. The tie-breaking rule is unknown; we avoided round numbers.
- **Visibility.** 45% of our offers at 89.8% will be noticed in the stage-1 results. If it works, the other teams will copy it in stage 2, and the abandoned pool will be shared or competed away. We bank it while it is unique.

8 Plan for stage 2

The stage-1 results give us the realized market: who each applicant borrowed from, who defaulted, and – we hope – the competitors' rates. The plan is (i) to infer each competitor's actual pricing function from the results (regressing their rate on X and the signals if rates are disclosed, otherwise from who won which applicant), (ii) to measure the realized curse on our own loans segment by segment and compare it with the simulation, (iii) to replace the twelve archetypes with the two estimated competitors and re-optimize, and (iv) only then to consider aggressive moves: undercutting where a competitor is shown to overprice, moving the net against theirs, and re-opening the 10–25% PD band if the measured curse turns out smaller than simulated. Everything in this note is reproducible, so stage 2 is a change of inputs, not a rewrite.

9 Files and how to reproduce

Everything lives in the project folder.

- `run_round1.py` – the whole pipeline. It regenerates the submission in about two minutes and prints every table of this note:

```
python run_round1.py --no-sex --net 0.898
```

- `lending_game/` – the package: `features.py`, `models.py`, `simulator.py` (rules of the game), `competitors.py` (archetypes, scenarios), `strategy.py` (our rule, optimizer).
- `analysis/` – `robustness_round1.py` (thresholds, scenario weighting) and `net_rate_scenarios.py` (net rate).
- `output/round1/`:
  - `rates_round1.csv` – the submission;
  - `strategy_round1.json` – all parameters of the rule and of the model;
  - `diagnostics_round1.md` – every table printed by the pipeline;
  - `new_applications_scored.csv` – our PD and rate per applicant, to diagnose stage 1 later;
  - `oos_probas_sex0.npz` – out-of-sample PDs of the four models on the past loans.